

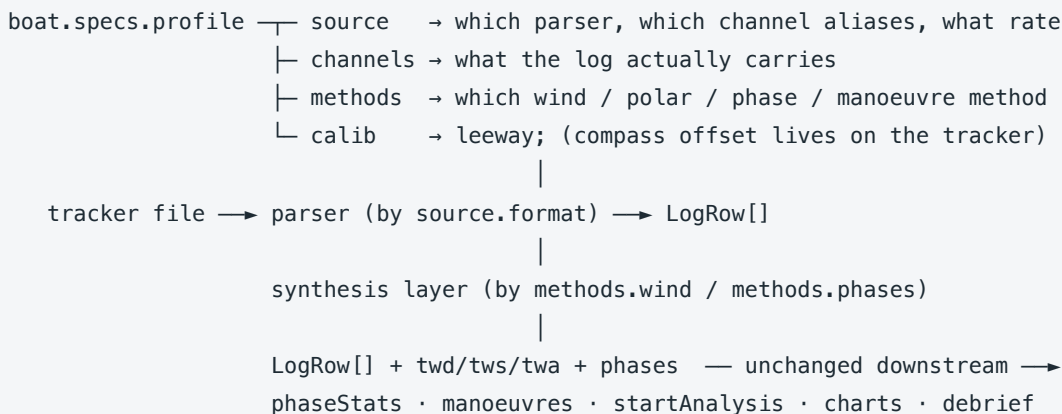
Adapting SSA for dinghy sailing — the build plan

Step-by-step, in dependency order. The spine is a **per-boat profile** that declares what the boat's log looks like and which methods to use on it; everything else hangs off that.

Background, and the evidence behind each choice: [dinghy-gps-prior-art-and-twd-2026-09.md](#) (incl. Part 4, field-validated against two real Atlas tracks) · [multi-boat-squad-research-2026-09.md](#) · [one-product-capability-profiles-2026-09.md](#)

Ground rule for every step: `npm run verify` stays green, and each step ships something usable on its own. No step is allowed to leave the N76 worse off.

0. The idea in one diagram



One profile decides everything variable. Nothing downstream learns a new concept.

1. The per-boat profile (*the keystone — do this first*)

Today `boat.specs.log_profile` holds per-boat channel aliases. Widen it into the full profile rather than adding a second competing concept.

```
// src/lib/boatProfile.ts – extends the existing logProfile.ts
export type LogFormat =
  | 'expedition' | 'flat-csv'           // existing N76 paths
  | 'vakaros-csv' | 'vakaros-vkx'
  | 'velocitek-vcc' | 'sailmon-csv' | 'gpx' | 'tractrac'

export interface BoatProfile {
  class?: string                // 'ILCA 7' | '49er' | 'Northstar 76'
  source: {
    format: LogFormat | 'auto'    // 'auto' = sniff from content
    aliases?: Partial<Record<LogField, string[]>> // ← today's log_profile
    rateHz?: number | 'auto'
  }
  channels?: LogField[]          // filled on first ingest, not by hand
  methods: {
    windDirection: 'measured' | 'derived-cog' | 'derived-fleet' | 'model' | 'manual'
    windSpeed:      'measured' | 'model' | 'manual'
    polar:          'measured' | 'published' | 'learned' | 'none'
    phases:         'event-file' | 'derived'
    manoeuvres?: { minSogKn?: number; targets?: { tack: number; gybe: number } }
  }
  calibration?: { leewayDeg?: number }
  classRules?: { electronicsWhileRacing?: boolean } // ILCA – see prior-art §3
}
```

Two profiles to seed:

	Northstar 76	ILCA / 49er
source.format	expedition	vakaros-csv
methods.windDirection	measured	derived-cog
methods.windSpeed	measured	model
methods.polar	measured	learned
methods.phases	event-file	derived

Why the compass offset is NOT here: devices move between boats. It belongs on the tracker record (step 5), because it is a property of the *instrument*, not the hull.

- **Files:** `src/lib/boatProfile.ts` (new), `src/lib/logProfile.ts` (absorbed).
- **Storage:** `boat.specs.profile` — JSONB, **no migration**, same pattern as `sails.specs` and today's `log_profile`.
- **Acceptance:** the N76 profile written explicitly reproduces today's behaviour byte-for-byte; the existing fixture tests pass untouched.
- **Risk:** scope creep into a config language. Keep it declarative — a profile *selects* a method, it never parameterises one beyond the fields above.

2. Channel provenance

Before any channel is synthesised, every channel needs to say where it came from. Retrofitting this later is the expensive version, and the wind work needs it regardless (prior-art §8, §13d).

```
export type Provenance = 'measured' | 'derived' | 'modelled' | 'unavailable'
// carried per channel per row-range, not per row – a parallel sparse map,
// NOT new keys on LogRow (which is Record<string, number|null>).
```

- **Files:** `src/lib/provenance.ts` (new); consumed by charts and the video overlay.
- **Acceptance:** a chart fed a `derived` TWD renders visibly differently from a `measured` one, and an `unavailable` range renders as a gap — never as zero.
- **Why now:** this is the single mechanism that lets one app serve both boats (architecture note §4), and it is what stops a derived number being argued about in a debrief as though it were measured.

3. The Vakaros CSV parser

The narrowest useful slice: one real file in, `LogRow[]` out.

```
timestamp,latitude,longitude,sog_kts,cog,hdg_true,heel,trim
```

Non-negotiables, all learned the hard way from the two real files (prior-art §15):

- **strip** `\r` — CRLF endings, or the last column silently vanishes
- **timestamps carry an explicit offset** (`+0100`) — parse it, and take the session date from the first timestamp, **never from the filename**
- **no device identity in the file** — do not try to infer the boat from content
- **rate varies per device** (2 Hz and 10 Hz seen in one session) — detect it
- reject implausible attitude (one unit logged heel $-173^\circ \dots +133^\circ$)
- map to canonical names: `sog_kts`→`sog`, `cog`→`cog`, `hdg_true`→`hdg`, `heel`, `trim`→`trimAngle`, and `sog_bsp` **only with** `provenance: 'derived'` — a dinghy has no paddlewheel, and this is exactly the kind of silent equivalence CLAUDE.md exists to prevent
- **Files:** `src/lib/parsers/vakarosCsv.ts`, fixtures in `src/lib/__tests__` .
- **Acceptance:** both real files parse to the expected row counts (22,812 and 104,617), correct UTC instants, and `trim` present and finite.
- **Then:** `gpx.ts` (universal fallback), `vakarosVtx.ts` (binary, open spec, gives the quaternion), `velocitekVcc.ts`, `sailmonCsv.ts` .

4. Derived phases — segmentation

No KND event file, so `buildPhases` gains a track-derived source: steady segments where heading/COG is settled and the boat is moving.

Working parameters from the real data: ≥ 12 s, circular resultant > 0.99 , `sog` > 2 kn, `|heel|` $< 60^\circ$ — which yielded 84 and 48 segments (95 and 71 minutes of steady sailing) across a 3-hour session.

- **Files:** `src/lib/wind/segment.ts`, wired into `buildPhases.ts` behind `methods.phases === 'derived'` .
 - **Acceptance:** segments never span a manoeuvre; total steady time is a plausible fraction of moving time.
 - **Note:** segmentation and wind estimation are mutually recursive — run as EM (segment → estimate → re-segment). Do not try to finish one before the other.
-

5. Trackers, identity, and compass calibration

```
-- migration 0070
CREATE TABLE trackers (
  id UUID PRIMARY KEY, team_id UUID NOT NULL REFERENCES teams(id),
  kind TEXT NOT NULL, label TEXT, serial TEXT,
  hdg_offset_deg REAL, hdg_offset_source TEXT, hdg_offset_at DATE
);
CREATE TABLE tracker_assignments (
  tracker_id UUID REFERENCES trackers(id), boat_id UUID REFERENCES boats(id),
  valid_from DATE NOT NULL, valid_to DATE
);
```

Identity rules (multi-boat §3b, and now settled by evidence):

1. **Primary: identity comes from auth.** The sailor uploads logged in as themselves; `memberships` already says which boat, time-bounded.
2. **Secondary:** the tracker registry, for the coach's bulk drop.
3. **Never:** the filename. The two real files were `Miss Behavior 2 2-8-2026` and `Torvar's second 08-02-2026` — the same day in two different date formats, one naming a boat and one naming a session.
4. **Quarantine:** an unconfirmed track is viewable but contributes to **no** squad statistic.

Compass calibration (prior-art §17) goes here: solve `COG - HDG` on opposite upwind tacks; the tack-independent part is the device offset, the part that flips sign is leeway. Store the offset on the tracker.

*Only valid on **non-tidal** days — a cross-wind current is algebraically indistinguishable from a compass offset. Calibrate in Palma, carry the offsets to the Solent. Record `hdg_offset_at` so a stale calibration is visible.*

- **Acceptance:** re-deriving the offsets from the two Palma files gives $\approx -5^\circ$ and $\approx -15^\circ$, and applying them collapses the boats' heading-derived TWD disagreement from 10.2° toward the 0.2° that COG achieves.

6. The wind synthesis layer

```
// src/lib/wind/estimate.ts
combine(sources: WindTrack[]): WindTrack // additive, prior-art §13b
// TWD = model baseline + track deviation - current correction
// TWS = model baseline (+ polar residual later)
```

Build in this order, and **use COG, not heading** (prior-art §16):

1. **L0 model baseline** — plug in the existing weather stack. Already built, and already better than Njord's OpenWeather Time Machine: Open-Meteo historical, 15-minutely in Europe, ICON-D2 at 2 km. *Watch the per-location billing — snap to a coarse grid.*
 2. **L2 symmetry fit** over steady segments → absolute TWD anchors.
 3. **The quality gate.** Non-optional: both tacks present upwind ≥ 2 min, score above threshold, else `unmeasured`. Measured effect on real data: **median error $75^\circ \rightarrow 7^\circ$** .
 4. **L3 relative tracking** between anchors (COG deviation on one tack, with SOG as the lift-vs-pinch discriminator).
 5. **L4 current solve** — only once there is a tidal venue to test on. Untested so far; Palma has no tide.
- **Acceptance:** on the Palma session, pooled TWD within $\sim 7^\circ$ of the model in gated windows, and windows without balanced tack data return `unmeasured`.

- **Read first:** SAP's `com.sap.sailing.windestimation` (Apache 2.0) — the only readable production implementation.
-

7. Learned polars

No dinghy class ships a polar. Bootstrap from published class target angles, then refine from the boat's own data once TWD is stable — a second EM loop, improving across a season.

- `polarCalc` gains a **provenance flag** so nothing downstream mistakes a learned polar for a measured one.
 - Gate on `methods.polar === 'learned'`.
 - **Note:** `polars` are currently applied at upload time. Njord has the same design and it is a documented pain point in their FAQ — make re-deriving a polar re-run cheaply rather than requiring re-upload.
-

8. The multi-boat scope

```
-- migration 0071
CREATE TABLE training_days (
  id UUID PRIMARY KEY, team_id UUID NOT NULL REFERENCES teams(id),
  date DATE NOT NULL, venue TEXT, lat DOUBLE PRECISION, lon DOUBLE PRECISION,
  tz_offset_minutes INTEGER,
  UNIQUE (team_id, date, venue)
);
ALTER TABLE sessions ADD COLUMN training_day_id UUID REFERENCES training_days(id);
```

Sessions keep `UNIQUE (boat_id, date)` and every existing FK. The new row is where the squad-level things attach.

- **Name check:** `public.events` is the **audit log**, not a regatta. Do not reuse the word.
 - Then: resample all boats in a day to a common grid (**coarsest contributing rate wins** — 2 Hz, not 10 Hz, when those are the two present).
 - Then: **re-run the wind estimator pooled across the squad**. This is the point of the whole exercise — pooling rescued windows where a single boat's fit failed (140° → 266° on real data).
-

9. The coach product

Only now is the app worth selling to a squad.

1. **Drill segmentation** on the training day — transit, tuning run, two-boat speed test, line-up, start practice, racing. Auto-propose, coach corrects. Reuse the tagger's vocabulary and lead/lag model; a drill spans boats, so it lives on `training_days`.
 2. **The two-boat speed test, protocol-aware** — the differentiator. Gain rate in metres/minute forward and sideways, **shift-compensated**, with the windward/leeward swap as the control across 3–4 runs, and an honest significance statement. Nobody does this for GPS-only boats; Deckman does it for instrumented yachts.
 3. **Line-ups** — 3+ boats ranked, high vs fast decomposed.
 4. **Squad debrief** — one timeline, N boats, media attached.
-

10. Fleet and event data

TracTrac when access lands. It arrives with **identity already solved** (an event roster), so it is the cheapest route to a full fleet and to the ~1° wind accuracy that five-plus boats buy. Existing notes: `docs/trac-trac-integration-research.md`.

Order, and what each phase is worth

phase	steps	ships
A — foundations	1, 2, 3	a dinghy track uploads, renders on the map, with honest provenance
B — analysis	4, 6 (L0–L3)	TWA, VMG, modes, manoeuvres, start-line bias — the existing app, alive on a dinghy
C — squad	5, 8	a coach uploads six boats and gets one wind and one day
D — product	9	drills and the speed test: the reason to switch tools
E — fleet	10, 7, 6-L4	regattas, learned polars, tidal venues

Phases A–C are mostly plumbing; D is the product. Worth saying out loud before starting so the plumbing does not feel like drift.

Risks worth naming

- **EM loops not converging** (segments ↔ wind, polar ↔ wind). Mitigation: seed from the model prior, cap iterations, and fail to `unmeasured` rather than to a wrong answer.
- **The per-boat profile becoming a config language.** Keep it declarative.
- **Provenance retrofitted late.** It is step 2 for a reason.
- `sog` **silently becoming** `bsp`. Mark it derived at the parser boundary and add it to CLAUDE.md's trap list the day it ships.
- **Untested against tide.** Every current claim in the research is theory until a Solent or Hyères day exists. Do not ship current correction off the back of a Palma dataset.